Stage-5 实验报告

计02 刘明道 2020011156

Step-11 & Step-12 实验内容

在这个 Stage 要支持数组的声明、下标、初始化和传参。主要实验内容如下

语法解析

增加和修改 AST 节点

新增数组下标和数组初始化列表的 AST 节点

```
class Subscription(Expression):
    def __init__(self, base: Expression, index: Expression):
        super(Subscription, self).__init__("subscription")
        self.base = base
        self.index = index
    ...

class InitializerList(Node):
    def __init__(self, initializer_list: List[IntLiteral]):
        super().__init__("initializer_list")
        self.initializer_list = initializer_list
        self.value = [x.value for x in initializer_list]
    ...
```

修改 `Declaration` 节点，使其支持数组的声明。在构造函数允许传入一个 `IntLiteral` 列表，这就表示声明的变量是个数组，并在构造过程中数组维度的合法性：

- 对于函数参数声明，允许第一维为空
- 否则要求每个维度都是正数

具体代码如下

```
class Declaration(Node):
    def __init__(
        ...
        array_dim: Optional[List[IntLiteral]] = None,
    ) -> None:
        ...
        if isinstance(self, Parameter):
            if array_dim is not None:
                for idx, dim_literal in enumerate(array_dim):
                    if idx == 0 and dim_literal is NULL:
                        continue
                    if dim_literal.value <= 0:
                        raise DecafBadArraySizeError()
        else:
            if array_dim is not None:
                for dim_literal in array_dim:
```

```
                    if dim_literal.value <= 0:
                        raise DecafBadArraySizeError()
        self.array_dim = array_dim or NULL
```

修改文法

文法上的修改主要是参考规范加入数组有关文法。

```
def p_subscription(p):  # 数组下标
    """
    postfix : postfix LSquBr expression RSquBr
    """
    p[0] = Subscription(p[1], p[3])

def p_array_dim_single(p):  # 声明数组维度（一个维度）
    """
    array_dim_decl : LSquBr Integer RSquBr
    """
    p[0] = [p[2]]

def p_array_dim_component(p):  # 递归地定义用于声明的数组维度
    """
    array_dim_decl : array_dim_decl LSquBr Integer RSquBr
    """
    p[1].append(p[3])
    p[0] = p[1]

def p_array_decl(p):  # 仅声明数组
    """
    declaration : type Identifier array_dim_decl
    """
    p[0] = Declaration(p[1], p[2], array_dim=p[3])

def p_array_decl_init(p):  # 声明数组并初始化
    """
    declaration : type Identifier array_dim_decl Assign initializer_list
    """
    p[0] = Declaration(p[1], p[2], p[5], array_dim=p[3])

def p_array_dim_param_first_dim_empty(p):  # 用于传参的空数组维度
    """
    array_dim_param : LSquBr RSquBr
    """
    p[0] = [NULL]

def p_array_dim_param_first_dim_not_empty(p):  # 用于传参的非空数组维度
    """
    array_dim_param : LSquBr Integer RSquBr
    """
    p[0] = [p[2]]

def p_array_dim_param_component(p):  # 递归地定义用于传参的数组维度
    """
    array_dim_param : array_dim_param LSquBr Integer RSquBr
    """
```

```
    p[1].append(p[3])
    p[0] = p[1]

def p_array_param(p):  # 数组类型的函数参数
    """
    parameter : type Identifier array_dim_param
    """
    p[0] = [Parameter(p[1], p[2], array_dim=p[3])]

def p_array_initializer_list_elem_single(p):  # 初始化列表元素的基础
    """
    initializer_list_elems : Integer
    """
    p[0] = [p[1]]

def p_array_initializer_list_elem_component(p):  # 初始化列表元素的递归定义
    """
    initializer_list_elems : initializer_list_elems Comma Integer
    """
    p[0] = p[1] + [p[3]]

def p_array_initializer_list(p):  # 初始化列表
    """
    initializer_list : LBrace initializer_list_elems RBrace
    """
    p[0] = InitializerList(p[2])
```

符号解析

`namer.py` 的符号解析过程主要做了如下修改

- 访问函数时，确定函数中声明了哪些局部数组，以及有哪些参数是数组，并将这一信息附加到相应的函数 AST 节点上。这些信息沿着 `Function -> FuncVisitor -> TACFunc -> SubroutineInfo` 进行传递，可用于生成 TAC 以及后端生成汇编代码。
- 访问声明时，如果是数组声明，则调用 `ArrayType.multidim` 生成相应的数组数据类型，赋给相应的 `VarSymbol` 。
- 由于现在有除了 `int` 之外的类型（数组类型），所以需要进行类型检查。给节点添加 `type` 属性，表示表达式的类型
  - 访问下标时，根据 `base` 类型计算出下标运算后的类型，赋给节点的 `type` 属性。由于下标是递归定义的，所以每次访问下标索引掉一层类型即可。在其他产生表达式的访问操作时相应配置 `type` 属性。
  - 接下来就是操作时检查类型一致性
    - 函数返回时，如果返回类型不是 `int` 则报错。
    - 二元运算左右表达式类型不一致报错
    - 三元运算 `then` 与 `otherwise` 类型不一致时报错

生成 TAC

生成 TAC 的过程主要进行了 ① 确定数组元素的地址，② 完成数组的初始化。

确定数组元素的地址

首先在 `FuncVisitor` 中实现类似于与 `la` `lw` `sw` 的三个访问方法，用于确定数组的地址，读写数组的元素

```
def visitLoadSymbolAddress(self, sym: VarSymbol):
    dst = self.freshTemp()
    self.func.add(LoadSymbolAddress(sym, dst))
    return dst

def visitLoadFromAddress(self, addr: Temp):
    dst = self.freshTemp()
    self.func.add(LoadWord(dst, addr, 0))
    return dst

def visitStoreToAddress(self, src: Temp, addr: Temp):
    self.func.add(StoreWord(src, addr, 0))
```

在访问过程中，对于数组标识符及其索引，使用 `addr` 属性计算其地址

- 访问数组标识符时，其地址置为调用 `FuncVisitor.visitLoadSymbolAddress` 生成 TAC 获得相应符号地址
- 访问数组索引时
  - 首先访问 `base` 和 `index` ，生成获得 `base` 地址和 `index` 值的 TAC

    ```
    sub.base.setattr('addr_only', True)
    sub.base.accept(self, mv)
    sub.index.accept(self, mv)
    assert sub.index.getattr('val')
    ```

    其中 `addr_only` 表示不需要生成从相应地址获取数据值的 TAC，这样可以省去一些无用代码
  - 接下来生成 TAC 计算当前索引对应地址的，将地址对应的变量存储到 `addr` 属性中

    ```
    # determine expression address
    base_offset_temp = sub.base.getattr('addr')
    offset_temp = mv.visitLoad(sub.getattr('type').size)
    mv.visitBinarySelf(tacop.BinaryOp.MUL, offset_temp, sub.index.getattr('val'))
    mv.visitBinarySelf(tacop.BinaryOp.ADD, offset_temp, base_offset_temp)
    sub.setattr('addr', offset_temp)
    ```

  - 如果需要取值，则从刚才计算到的地址中取值

    ```
    # determine expression value
    if sub.getattr('addr_only') is None:
        sub.setattr(
            'val',
            mv.visitLoadFromAddress(offset_temp)
        )
    ```

- 访问函数调用时，如果参数类型为数组，则将将储存地址的临时变量绑定为参数

```
if isinstance(arg_expr.getattr('type'), ArrayType):
    assert arg_expr.getattr('addr')
    param_temp += [arg_expr.getattr('addr')]
else:
    ...
```

- 访问赋值时，如果左操作数是下标运算，则先访问下标获得其地址，然后将右操作数的值保存到相应地址。

```
if isinstance(expr.lhs, Subscription):
    expr.lhs.setattr('addr_only', True)
    expr.lhs.accept(self, mv)
    mv.visitStoreToAddress(
        rhs_val,
        expr.lhs.getattr('addr')
    )
    return
...
```

完成数组的初始化

- 对于函数内声明的数组
  - 如果初始化表元素比数组展开后的元素少，则调用 `fill_n` 函数将后续元素补 0

    ```
    # Call `fill_n` to clear array if not all element are defined in initializer list
    addr = mv.visitLoadSymbolAddress(sym)
    value_temp = mv.visitLoad(0)
    dummy = mv.freshTemp()
    if len(decl.init_expr.value) < sym.type.element_count:
        const_temp = mv.visitLoad(sym.type.size)
        mv.visitCall(
            mv.ctx.getFuncLabel(mv.lib_function["fill_n"].ident.value),
            dummy,  # dummy, store return value
            [addr, value_temp, const_temp]
        )
    ```

  - 对于初始化表中的元素，逐一向数组的对应位置赋值

    ```
    # Initialize elements specified in initializer list
    const_temp = mv.visitLoad(sym.type.full_indexed.size)
    for val in decl.init_expr.value:
        mv.visitLoad(val, value_temp)
        mv.visitStoreToAddress(value_temp, addr)
        mv.visitBinarySelf(tacop.BinaryOp.ADD, addr, const_temp)
    ```

- 对于全局数组，其初始化直接通过汇编代码完成

生成 RISC-V 汇编代码

全局数组的初始化

对于有初始化的全局数组，将初始化的值放置在相应的 `.data` 段，初始化表元素少于数组元素的，末尾全部初始化为 0

```
class RiscvAsmEmitter(AsmEmitter):
    def __init__(
        ...
    ) -> None:
        ...
        # Global array variable
        for decl in filter(lambda x: isinstance(x.init_expr, InitializerList), globalDecls):
            self.printer.printDATAWords(
                decl.ident.value,
                decl.init_expr.value + [0] * (decl.getattr('type').element_count -
len(decl.init_expr.value))
            )
```

数组传参支持

调整栈帧结构

首先，为了方便，我们将 Stage-4 中定义的栈帧结构中添加局部数组

```
Arg (len - 1)
...
Arg 8
------------- `self.nextLocalOffset`
SPILL END
...
SPILL BEGIN
RA
CALLEE END
...
CALLEE BEGIN
LOCAL ARRAY BEGIN
...
LOCAL ARRAY END
------------- SP
```

并对 callee save 保存，`ra` 保存等代码进行了相应的修改。

分配局部数组位置

在构造 SubRoutineInfo 时，标记参数数组，并为函数内声明的数组分配栈帧上的位置

```
class SubroutineInfo:
    def __init__(self, func: TACFunc) -> None:
        ...
        # split different kinds of arrays
        self.localArrays = func.local_arrays
        self.argArrays = {var: idx for var, idx in func.param_arrays}

        # stack space for all local arrays
        self.array_offsets: Dict[VarSymbol, int] = {}
        alloc_ptr = 0
```

```
        for var in self.localArrays:
            self.array_offsets[var] = alloc_ptr
            alloc_ptr += var.type.size
        self.localArraySize = alloc_ptr
```

加载数组地址

对于符号地址的加载，分为以下几种情况

- 全局变量，和 Stage-4 一样，生成 `la` 指令
- 局部变量
  - 如果是函数参数，直接进行从传入的参数复制值
  - 如果是函数内声明的数组，那么数组在栈帧上，从 `self.ctx.array_offset` 进行复制偏移量然后与栈指针的值相加

```
def visitLoadSymbolAddress(self, instr: LoadSymbolAddress) -> None:
    if instr.symbol.isGlobal:  # Global symbols use `la` directly
        self.seq.append(Riscv.LoadLabel(instr.dsts[0], instr.symbol.name))
    else:
        assert isinstance(instr.symbol.type, ArrayType)  # Only ArrayType can invoke this
instruction
        if instr.symbol in self.ctx.localArrays:  # Arrays assigned in stack
            self.seq.append(Riscv.AddImm(Riscv.SP, instr.dsts[0],
self.ctx.array_offsets[instr.symbol]))
        elif instr.symbol in self.ctx.argArrays:  # Arrays passed from parameter
            self.seq.append(Riscv.Move(instr.dsts[0],
self.ctx.argTemps[self.ctx.argArrays[instr.symbol]]))
        else:
            raise ValueError(instr.symbol)
```

其中 `self.ctx` 为函数对应的 SubRoutineInfo 对象； `AddImm` 为这个 Stage 新实现的指令，实际上就是 `RISC-V` 指令 `addi` 。

思考题

Step-11

1. C 语言规范规定，允许局部变量是可变长度的数组（Variable Length Array，VLA），在我们的实验中为了简化，选择不支持它。请你简要回答，如果我们决定支持一维的可变长度的数组(即允许类似 `int n = 5; int a[n];` 这种，但仍然不允许类似 `int n = ...; int m = ...; int a[n][m];` 这种)，而且要求数组仍然保存在栈上（即不允许用堆上的动态内存申请，如`malloc`等来实现它），应该在现有的实现基础上做出那些改动？
   - 首先在文法中加入相应的支持
   - 调整栈帧布局和栈上的偏移方式

     将前述栈帧布局调整为

```
Arg (len - 1)
...
Arg 8
------------- `self.nextLocalOffset`
SPILL END
...
```

```
SPILL BEGIN
RA
CALLEE END
...
CALLEE BEGIN
LOCAL ARRAY BEGIN
...
LOCAL ARRAY END
------------- [FP] ([SP] after initial adjustment)
LOCAL DYNAMIC ARRAY BEGIN
...
LOCAL DYNAMIC ARRAY END
------------- [SP]
```

在可变长度数组变量进入作用域时，生成指令计算数组占用的空间，然后将栈指针下拉相应空间；在相应可变长度数组的变量离开作用域时将栈指针收起相应长度。

此时，由于栈指针反复动态调整，随时使用栈指针确定变量在栈上的位置，以及确定数组相对栈指针的位置可能会略有复杂。为此可以在函数调用的首次下拉栈指针（这一下拉量是固定的）后，使用 `fp` 寄存器来记录当前栈指针的位置（如图中 `[FP]` 所示），而在后续翻译 `LoadSymbolAddress` 指令以及调用 `emitLoadFromStack/emitStoreToStack` 时，对于栈上的变量和动态分配额数组使用相对 `fp` 的偏移量来确定地址。

Step-12

1. 作为函数参数的数组类型第一维可以为空。事实上，在 C/C++ 中即使标明了第一维的大小，类型检查依然会当作第一维是空的情况处理。如何理解这一设计？

   C 语言在编译期并不对数组越界进行检查，数组声明时的大小用于分配相应的内存空间以及计算数组在索引时相对起始地址的偏移量。而在数组作为函数参数时，被调用的函数并不需要为这些数组参数分配内存空间，而只需要计算其相对首地址的偏移量。计算偏移量不需要第一维大小，但需要后续维度的偏移量，因此第一维大小被编译器类型检查忽略，但后续维度大小必须匹配。